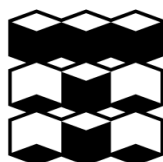


МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное автономное образовательное

учреждение высшего образования

**НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ
ТОМСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ**



Инженерная школа информационных технологий и робототехники

Отделение автоматизации и робототехники

Направление 15.04.06 «Мехатроника и робототехника»

«Сравнение алгоритмов планирования маршрута с объездом статических
препятствий на мобильном роботе Robotino»

Отчет по лабораторной работе № 2

по дисциплине «Мобильные роботы»

Выполнил: студент

гр. 8ЕМ51

(подпись)

(дата)

Лащёнов А.Д.

Проверил: старший

преподаватель ОАР ИШИТР

(подпись)

(дата)

Поберезкин Н.

Оглавление

Цель работы	3
Задание	3
Исходные данные	3
Ход работы	4
Детекция границ поля и построения глобальной системы координат .	4
Определение местоположения и ориентации робота в глобальной системе	5
Детектирование препятствий	7
Построение маршрута движения	10
Подключение к роботу и его отправка	12
Проведение тестового заезда	15
Вывод.....	20

Цель работы

Реализация и проведение сравнительного анализа не менее трех алгоритмов планирования маршрута с объездом статических препятствий.

Задание

Требуется реализовать движение робота из точки “А” в точку “Б” с алгоритмом объезда препятствий с выводом координат робота и отображением пройденного пути. Реализовать необходимо не менее трёх алгоритмов планирования маршрута.

Исходные данные

На поле 2200х2200 мм находится 5 статических препятствий (вид и размеры препятствия на выбор исполнителя). Над полем расположена видеокамера.

Ход работы

Для выполнения поставленной задачи необходимо:

1. Провести детектирование границ поля и определение глобальной системы координат;
2. Провести определение местоположения и ориентации робота в глобальной системе;
3. Провести детектирование препятствий;
4. Разработать алгоритм поиска маршрута;
5. Разработать алгоритм движения Robotino.

Детекция границ поля и построения глобальной системы координат

Для определения границ поля было принято решение вручную указывать точки на видео потоке, которые определяют углы поля. Так как поля – прямоугольник, то было принято решение призвать глобальную систему к углам поля. Первая точка (угол) определяет начало системы координат. Вторая точка определяет направление оси абсцисс. Последняя точка определяет направление оси ординат (см. рис. 1).

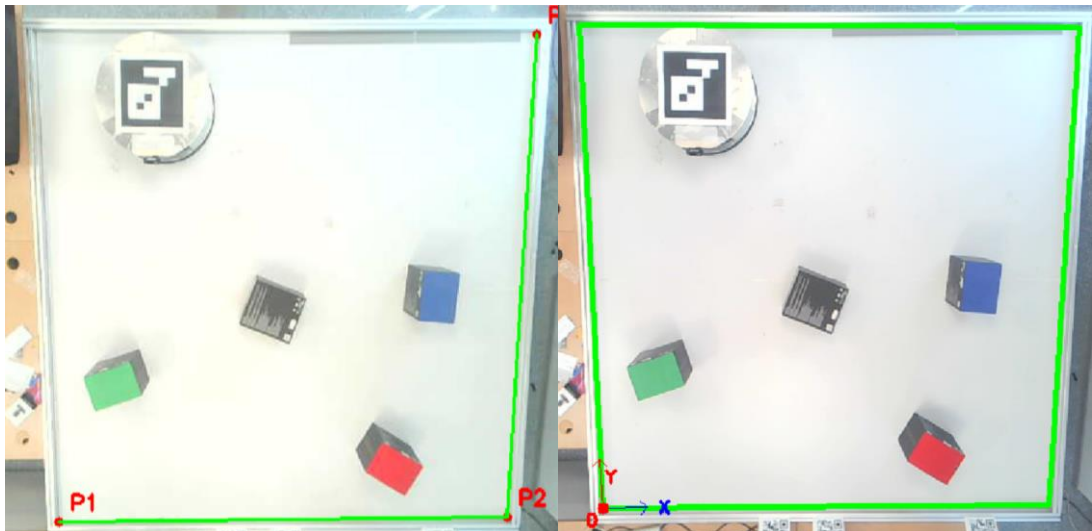


Рисунок 1 – Определение границ поля и глобальной системы координат

Определение местоположения и ориентации робота в глобальной системе

Локализация мобильного робота осуществляется методом визуальной навигации с использованием двумерных маркеров ArUco. Данный подход обеспечивает высокую точность определения как позиции, так и угла поворота платформы в реальном времени.

Процесс вычисления глобальных координат и ориентации включает следующие этапы:

1. Детектирование маркера. Каждый кадр с камеры обрабатывается детектором `cv2.aruco.ArucoDetector` (используется словарь `DICT_6X6_50`). При успешном распознавании извлекаются пиксельные координаты четырех углов маркера и вычисляется его геометрический центр.

2. Перспективное преобразование. Для перехода от искаженной пиксельной системы координат камеры к ортогональному виду сверху применяется матрица перспективного преобразования (M), рассчитанная на этапе калибровки по четырем угловым точкам рабочего поля из файла конфигурации.

3. Перевод в метрическую систему. Спроецированные координаты центра маркера масштабируются с использованием калибровочных коэффициентов px_x и px_y (количество пикселей на метр по осям X и Y). В результате робот получает свои глобальные координаты (X, Y) в метрах относительно начала координат поля (верхний левый угол преобразованного изображения).

4. Определение ориентации (курса). Угол поворота робота вычисляется на основе координат углов маркера в проекции сверху (`pts_top`). Базовый угол наклона маркера определяется через арктангенс разности координат его вершин. С учетом конструктивной особенности крепления (ось X робота конструктивно совпадает с осью Y маркера), к полученному значению применяется постоянная поправка $-\pi/2$ радиана.

5. Валидация положения. Центр робота проверяется функцией `cv2.pointPolygonTest` на принадлежность многоугольнику рабочего поля. Это позволяет отфильтровать ложные срабатывания, если в кадр случайно попал посторонний ArUco-маркер за пределами трассы.

6. Полученные метрические координаты центра и угол ориентации далее передаются в модуль кинематики для расчета вектора скоростей и в алгоритм планирования для оценки текущего состояния системы.

На рисунке 2 изображен робот с отрисовкой.

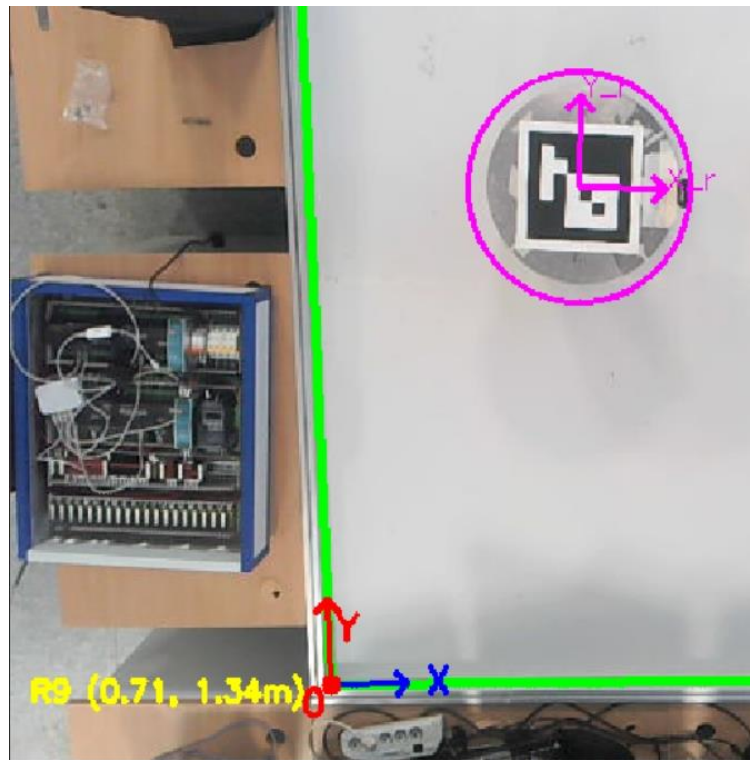


Рисунок 2 – Определение системы координат робота и его координат в глобальной системе

Детектирование препятствий

Обнаружение препятствий в системе выполняется на основе цветовой фильтрации в цветовом пространстве HSV¹.

Применение данного подхода обеспечивает устойчивость алгоритма к изменениям освещенности внутри помещения, что критически важно для стабильной работы робота. Тем не менее, для минимизации влияния теней и камерального шума применяется комплекс морфологических операций и геометрических проверок

Алгоритм построения маски и извлечения данных о препятствиях включает следующие шаги:

1. HSV-фильтрация. Исходный кадр преобразуется из пространства BGR в HSV. К кадру последовательно применяется функция `cv2.inRange` для заданных цветовых диапазонов (например, красный, зеленый, синий, черный), в результате чего формируется бинарная маска потенциальных препятствий.

2. Исключение робота и проверка границ. Поскольку ArUco-маркер на роботе может быть ошибочно классифицирован как препятствие (особенно при детектировании черного цвета), на бинарную маску программно накладывается черная окружность в координатах центра робота, «стирая» его из поля поиска. Дополнительно центр каждого обнаруженного объекта проверяется функцией `cv2.pointPolygonTest` на принадлежность многоугольнику рабочего поля, что позволяет исключить ложные срабатывания за его пределами.

3. Морфологическая очистка. Для устранения мелкого шума и заполнения внутренних пустот в объектах к маске применяются операции открытия (`cv2.MORPH_OPEN` с ядром 5×5 , 2 итерации) и закрытия (`cv2.MORPH_CLOSE` с ядром 5×5 , 4 итерации).

¹ HSV HSV (англ. Hue, Saturation, Value — тон, насыщенность, значение) — цветовая модель, в которой координатами цвета являются: цветовой тон, насыщенность и яркость.

4. Выделение контуров и фильтрация по площади. С помощью функции `cv2.findContours` (с флагами `RETR_EXTERNAL` и `CHAIN_APPROX_SIMPLE`) извлекаются внешние границы объектов. Контуров, площадь которых меньше порогового значения `MIN_OBSTACLE_AREA` (300 пикселей), отбрасываются как незначительный шум.

5. Определение центра и перспективное преобразование. Для оставшихся контуров вычисляются пространственные моменты (`cv2.moments`) для нахождения центра масс в пиксельных координатах. Затем эти координаты преобразуются в вид сверху (в метры) с помощью матрицы перспективного преобразования `cv2.perspectiveTransform`.

6. Инфляция (раздувание) препятствий. На этапе планирования маршрута (`path_planner.py`) создается дискретная сетка проходимости. Ячейки, соответствующие препятствиям, а также границы поля, принудительно помечаются как непроходимые с учетом эффективного радиуса: суммы радиуса робота (`obs_radius_m`) и заданного безопасного отступа (`safety_margin`). Это гарантирует, что центр робота физически не сможет приблизиться к препятствию на опасное расстояние.

Для визуализации процесса и интерфейса оператора в рабочем окне последовательно применяются следующие методы библиотеки `opencv-python`:

- `cv2.circle` и `cv2.arrowedLine` – отрисовка центра робота, его локальной системы координат (ориентированной по положению `ArUco`-маркера) и глобальных осей координат поля (в нижнем левом углу).
- `cv2.line` – отображение желаемого маршрута (жёлтая линия), построенного алгоритмом планирования.
- Отрисовка доступной для робота области (зелёные точки), вычисленной алгоритмом поиска в ширину (BFS). Установка целевой точки в этой зоне гарантирует успешное построение маршрута.

- `cv2.putText` – вывод текстовых меток с текущими координатами робота и цели, а также названия активного алгоритма и длины пути.

Примерный вид визуализации рабочего окна с обнаруженными препятствиями и построенным маршрутом представлен на рисунке 3.

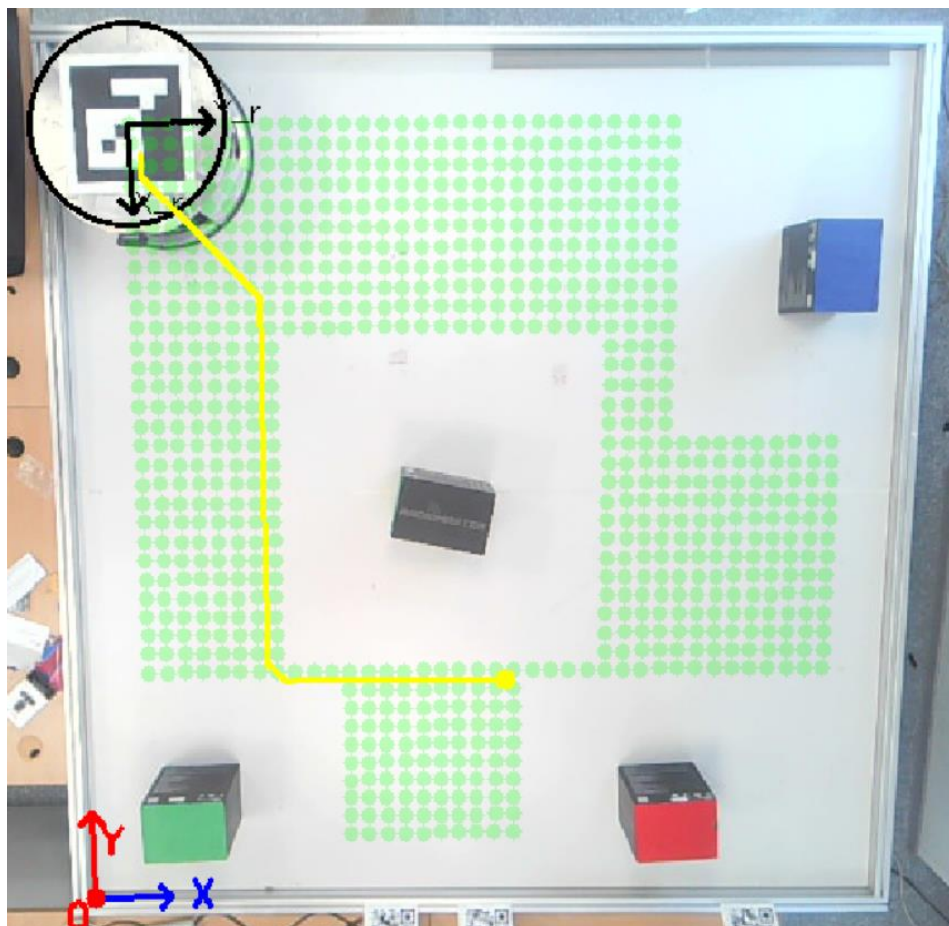


Рисунок 3 – Вид элементов визуализации

В рабочем окне имеются следующие элементы интерфейса:

- Отображение глобальной системы координат поля (нижний левый угол);
- Отображение центра робота и локальной системы координат робота (зависит только от положения ArUco-маркера);
- Отображение желаемого пути (жёлтая линия);
- Отображение доступной для робота области (зелёные точки).

Установка целевой точки в зелёной зоне гарантирует построение маршрута.

Построение маршрута движения

На основе данных детектирования препятствий формируется бинарная карта проходимости рабочего поля. Пространство дискретизируется на регулярную сетку с заданным размером ячейки (*cell_size*). Для обеспечения физической безопасности робота и компенсации погрешностей одометрии применяется операция расширения (*inflation*): зоны препятствий и границы рабочего поля программно увеличиваются на величину эффективного радиуса, равного сумме радиуса робота и заданного безопасного отступа (*safety_margin*). Координаты текущего положения робота и целевой точки преобразуются из метрической системы (метры) в индексы ячеек данной сетки.

В разработанной программе реализованы три алгоритма планирования пути, выбор между которыми осуществляется посредством изменения соответствующего параметра (*planner_type*) в конфигурационном файле *parameters.yaml*. А именно, реализованы следующие алгоритмы:

1. Алгоритм А (A-star)* является информированным алгоритмом поиска по графу, гарантирующим нахождение кратчайшего пути при использовании монотонной эвристики. Оценка приоритета узла вычисляется по функции (1):

$$f(n) = g(n) + h(n) \quad (1)$$

Где

$g(n)$ – фактическая накопленная стоимость пути от стартовой точки до узла n ;

$h(n)$ – эвристическая оценка расстояния от узла n до цели (в данной реализации используется евклидово расстояние).

Благодаря использованию эвристики алгоритм целенаправленно исследует пространство, посещая значительно меньше узлов по сравнению с алгоритмом Дейкстры, при этом сохраняя математическую оптимальность решения.

2. Двухнаправленный алгоритм Дейкстры в отличие от A^* , данный алгоритм является неинформированным и не использует эвристическую функцию для направления поиска. Оценка приоритета узла вычисляется исключительно по фактической накопленной стоимости: $f(n) = g(n)$. Механизм работы заключается в одновременном расширении двух фронтов поиска: одного от начальной позиции робота, другого – от целевой точки. Поиск завершается в момент пересечения фронтов, после чего два найденных фрагмента пути объединяются. Главное преимущество данного подхода – существенное сокращение пространства поиска (в среднем в 2–4 раза) по сравнению с классическим алгоритмом Дейкстры, при строгом сохранении гарантии нахождения кратчайшего возможного пути на дискретной сетке.

3. Жадный алгоритм поиска по первому наилучшему совпадению (Greedy Best-First Search) является информированным, но, в отличие от A^* , он полностью игнорирует стоимость уже пройденного пути. Функция оценки приоритета узла сводится исключительно к эвристике: $f(n) = h(n)$. На каждом шаге алгоритм выбирает для расширения тот соседний узел, который геометрически находится ближе всего к целевой точке. Это обеспечивает максимальную скорость построения маршрута при отсутствии препятствий. Однако у подхода есть критические недостатки: отсутствие гарантии оптимальности (найденный путь часто оказывается длиннее кратчайшего) и высокая уязвимость к локальным минимумам (например, при наличии U-образных препятствий алгоритм не сможет найти обходной путь, так как любое движение в сторону от цели увеличит значение эвристики и будет отвергнуто).

Архитектура планировщика включает механизмы обработки критических ситуаций (edge cases) для обеспечения отказоустойчивости системы:

1. Недостижимость цели: если целевая ячейка или все подходы к ней заблокированы препятствиями, алгоритм исчерпывает пространство поиска и возвращает пустой маршрут. В этом случае робот инициирует остановку, а в консоль выводится сообщение о невозможности построения маршрута.

2. Виртуальное застревание на старте: из-за погрешностей детектирования или агрессивных настроек безопасного отступа стартовая ячейка робота может быть ошибочно классифицирована как препятствие. Для предотвращения сбоя реализован алгоритм поиска в ширину (BFS), который сканирует окрестность робота (в радиусе нескольких ячеек) для поиска ближайшей свободной ячейки. При её обнаружении точка старта виртуально смещается в эту ячейку, позволяя построить корректный маршрут для выхода из зоны "виртуального застревания". Если свободных ячеек в заданном радиусе не обнаружено, система распознаёт физическое застревание и блокирует движение.

Результирующий массив индексов ячеек преобразуется обратно в метрические координаты, формируя дискретный опорный маршрут. В дальнейшем этот маршрут может быть подвергнут процедуре постобработки (например, сплайн-интерполяции) для обеспечения плавности управляющих воздействий.

Подключение к роботу и его отправка

Управление движением осуществляется отправкой HTTP-запроса на URL-вида “http://<IP>/data/omnidrive”.

Тело запроса представляет собой JSON-массив из трёх чисел: $[V_x, V_y, 0]$ (т.к. голономное движение). Функция “send_velocity” модуля “Robotino.py” формирует запрос и проверяет код ответа. При успешном ответе (статус “200”) команда считается исполненной. В случае сетевой ошибки выводится предупреждение.

Вектор скорости робота формируется на основе глобального пути с помощью релейного управления: при дистанции больше пороговой,

Полученный в глобальной системе координат рабочей области вектор скорости “velocity” должен быть пересчитан в систему координат, связанную с роботом. Следующий код (см. листинг 1).

Листинг 1 – Программный пересчёта скорости из глобальной системы в систему робота

```
# 1. Угол к следующей точке в системе поля
angle_to_wp = math.atan2(next_wp[1] - ry, next_wp[0] - rx)

V = SPEED_FAR if dist_to_wp >= SPEED_THRESH else SPEED_NEAR

# 2. Скорости в системе поля
Vx_field = V * math.cos(angle_to_wp)
Vy_field = V * math.sin(angle_to_wp)

# 3. Текущий угол маркера в поле
pts_top = robot_data["pts_top"]
marker_heading = math.atan2(pts_top[1][1] - pts_top[0][1], pts_top[1][0] - pts_top[0][0])

# 4. Угол ориентации робота
##: Ось X робота совпадает с Осью Y маркера.
## Ось Y маркера повернута на +90° (+pi/2) относительно оси X маркера.
psi = marker_heading - math.pi / 2

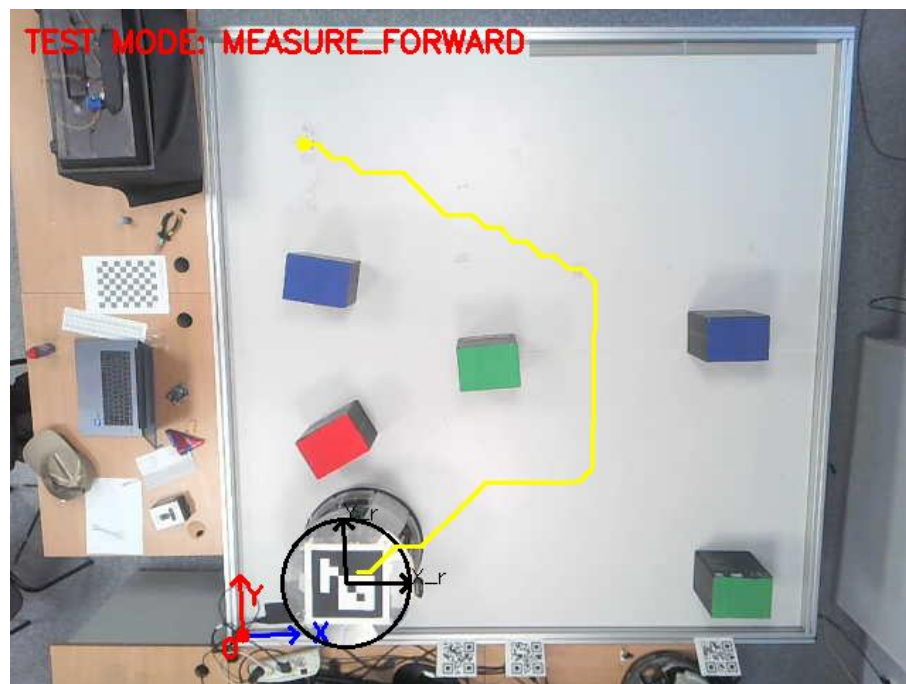
# 5. Поворот вектора скорости из поля в локальную систему робота
cos_psi = math.cos(psi)
sin_psi = math.sin(psi)
Vx_robot = Vx_field * cos_psi + Vy_field * sin_psi
Vy_robot = -Vx_field * sin_psi + Vy_field * cos_psi
```

Проведение тестов

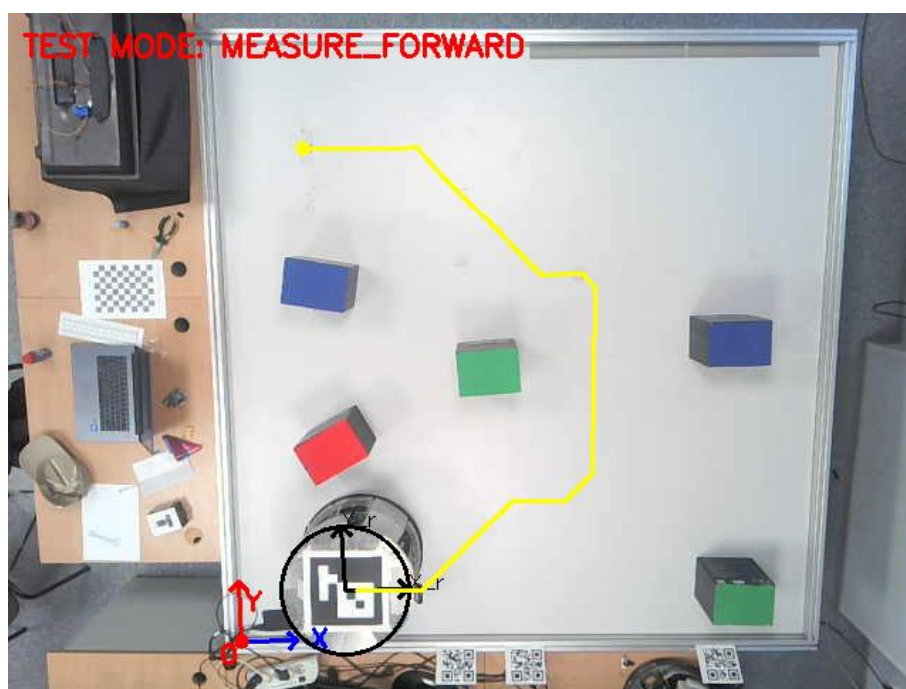
Для количественной оценки работы алгоритмов планирования и системы управления роботом в ходе автоматического теста фиксируются следующие метрики:

- Планируемая длина пути (Planned Path Length) — теоретическая длина маршрута, рассчитанная алгоритмом планирования от стартовой позиции до целевой точки. Характеризует оптимальность самого алгоритма поиска.
- Фактическая длина пути (Actual Path Length) — реальное расстояние, пройденное роботом, вычисленное как сумма евклидовых расстояний между последовательными зафиксированными позициями центра робота. Позволяет оценить физические потери на коррекцию курса.
- Время прохождения (Forward Time) — время, затраченное роботом на преодоление маршрута от старта до цели. Отражает динамическую эффективность и плавность траектории.
- Среднеквадратичное отклонение (MSE, мм²) — мера разброса фактических позиций робота относительно точек запланированного пути. Чем ниже значение, тем точнее робот физически повторяет геометрию маршрута.
- Коэффициент детерминации (R^2) — статистическая метрика (от 0 до 1), показывающая, насколько точно форма фактической траектории соответствует форме запланированной. Значение, близкое к 1.0, свидетельствует о высоком качестве системы управления.

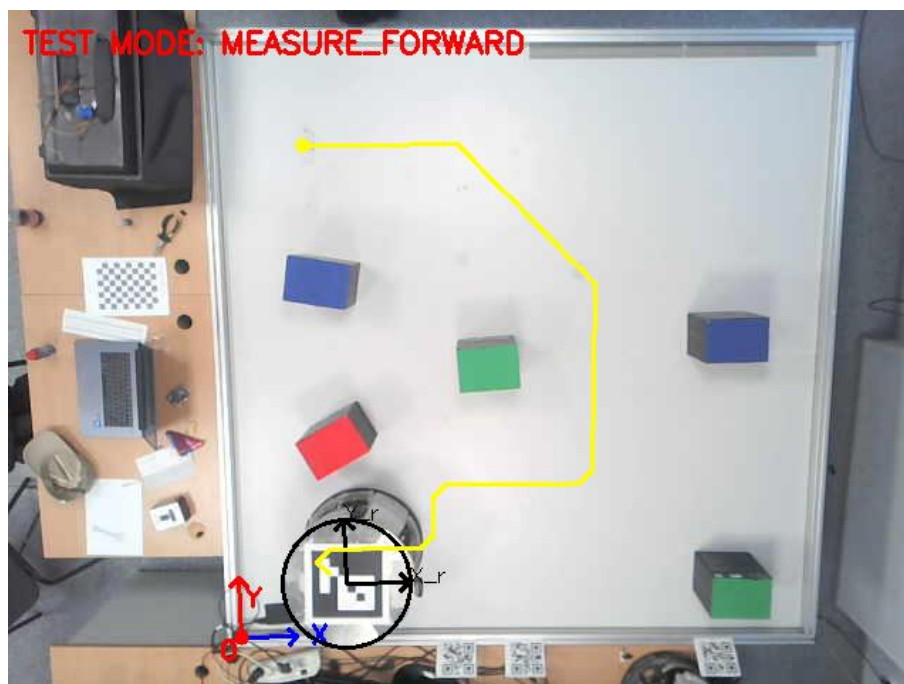
На рисунке 4 показано построение маршрута при движении из нижнего края карты в верхний.



a



6



в

а – алгоритм A*; б – двунаправленный алгоритм Дейкстры; в – жадный алгоритм

Рисунок 4 – Построение траектории от стартовой к конечной точке маршрута различными алгоритмами

Таблица 1 – Сводная таблица результатов экспериментов

Алгоритм	Планируемая длина, м	Фактическая длина, м	Время пути, с	MSE, мм ²	R ²
A* (A-star)	3,02	3,03	28,03	438,01	0,99
Двунаправленный Дейкстра	3,04	3,05	27,74	406,56	0,99
Жадный	3,19	3,37	29,75	375,65	0,99

На основе полученных экспериментальных данных можно сделать следующие заключения о работе исследуемых алгоритмов в условиях реального физического эксперимента:

1. Оптимальность пути (Анализ длин маршрутов)

Алгоритмы A* и Двухнаправленный Дейкстра продемонстрировали высокую оптимальность: их планируемые длины (3.023 м и 3.044 м соответственно) минимальны и практически совпадают. При этом фактическая длина пути почти идентична планируемой (разница составляет всего 4–5 мм), что говорит об отсутствии лишних маневров и высокой точности локального управления.

В отличие от них, Жадный алгоритм (Greedy) сформировал заметно более длинный маршрут (3.19 м), а фактическая длина оказалась еще больше (3.37 м). Разрыв между планом и фактом (около 18 см) указывает на то, что из-за субоптимальности геометрии пути системе управления приходилось постоянно корректировать курс, что физически увеличило пройденное расстояние. Это полностью подтверждает теоретический недостаток жадного алгоритма — отсутствие гарантии оптимальности.

2. Динамическая эффективность (Анализ времени)

Наилучшее время прохождения показал Bi-Dijkstra (27.74 с), чуть медленнее оказался A (28.03 с)*. Их оптимальные и предсказуемые траектории позволили роботу поддерживать высокую крейсерскую скорость без резких торможений.

Greedy оказался самым медленным (29.75 с). Несмотря на то, что теоретически он строит путь быстрее (за счет упрощенной эвристики), физическое движение по этому пути заняло больше времени из-за необходимости частых кинематических перестроений и снижения скорости в точках резкого изменения направления.

3. Качество отработки траектории (Анализ MSE и R^2)

Все три алгоритма показали выдающиеся значения коэффициента детерминации $R^2 > 0.998$. Это доказывает, что разработанная система кинематического управления (расчет скоростей и компенсация угла маркера) является высоконадежной и стабильной: робот четко следует заданному вектору скорости независимо от того, какой алгоритм сформировал путь.

Значения MSE для всех алгоритмов находятся в узком диапазоне (375–438 мм²). Если извлечь корень из этих значений, то среднее линейное отклонение робота от идеальной линии составит всего около 19–21 мм. Для мобильного робота диаметром 0.5 м на поле размером 2.2 м это является пренебрежимо малой погрешностью, обусловленной исключительно физическим проскальзыванием омниколес и погрешностями одометрии. Примечательно, что Greedy показал минимальный MSE, однако, как было показано выше, это достигнуто ценой значительного удлинения общего пути и времени движения.

Вывод

В результате выполнения лабораторной работы проведен сравнительный анализ трех алгоритмов планирования пути: A^* , двунаправленного Дейкстры и жадного алгоритма (Greedy).

На основе экспериментальных данных установлено, что для навигации в замкнутом полигоне со статическими препятствиями наиболее целесообразно использовать алгоритм A^* . Он обеспечивает наилучший баланс между оптимальностью маршрута и временем движения, минимизируя физические затраты робота.

Двунаправленный алгоритм Дейкстры выступает равноценной альтернативой: он гарантирует строгую математическую оптимальность без использования эвристик, демонстрируя сопоставимые с A^* показатели скорости (~ 27.7 с) и высокой точности следования траектории ($R^2 > 0.998$).

Жадный алгоритм, несмотря на высокую скорость вычислений и хорошую отработываемость системой управления ($R^2 = 0.9991$), формирует субоптимальные, извилистые траектории. Это приводит к значительному увеличению фактической длины пути и общего времени движения, что делает его применение нецелесообразным в задачах, где критична общая эффективность маршрута.

GitHub-репозиторий с программным кодом и файлом “README.md” расположен по ссылке: <https://github.com/heigan/Mobile-Robot-LB>

Поставленные цели лабораторной работы были достигнуты.